

1024HDC Zynq AXI Project Guide

Project Summary

This project implements a 1024-bit HDC processing block that:

- XORs a 1024-bit input vector with a 1024-bit bind vector
- Applies one of several permutation modes
- Returns a 1024-bit result

The compute core is wrapped with an AXI4-Lite slave so a Zynq Processing System (PS) can control it through memory-mapped registers.

Main RTL Files

- ``xor_permute_top.sv``
 - Core 1024-bit datapath
 - Input handshake, XOR stage, permutation stage, output stage
- ``permute_stage.sv``
 - Implements the permutation behavior
 - Mode ``00`` : reverse words
 - Mode ``01`` : rotate each 64-bit word
 - Mode ``10`` : rotate full 1024-bit vector
 - Mode ``11`` : passthrough/default
- ``hdc_axi_lite_wrapper.sv``
 - AXI4-Lite slave wrapper for Zynq PS access
 - Packs and unpacks 32-bit AXI registers into 1024-bit vectors

- `tb\_xor\_permute.sv`
- Simulation-only testbench
- Not used for synthesis

Data Widths

- Total vector width: 1024 bits
- Number of words: 16
- Bits per word: 64
- AXI register width: 32 bits
- Number of AXI words per vector: 32

AXI Register Map

- `0x000` Control register
 - Bit `0` : start
 - Bit `2` : clear done
- `0x004` Status register
 - Bit `0` : done
 - Bit `1` : busy
 - Bit `2` : input ready
 - Bit `3` : output valid
- `0x008` Permutation mode register
 - Bits `1:0` : mode
- `0x00C` Permutation parameter register
- `0x100` to `0x17C`
 - Input vector, 32 words of 32 bits
- `0x200` to `0x27C`

- Bind vector, 32 words of 32 bits
- `0x300` to `0x37C`
- Output vector, 32 words of 32 bits

Typical PS Transaction Flow

1. Write the 32 input words to the input register space
2. Write the 32 bind words to the bind register space
3. Write `perm\_mode`
4. Write `perm\_param`
5. Write control bit `0` to start processing
6. Poll status bit `0` until done is asserted
7. Read the 32 output words from the output register space
8. Write control bit `2` to clear the done flag

Vivado Block Design Steps

1. Create a new Vivado RTL project for the target Zynq device or board
2. Add these synthesis files:
 - `hdc\_axi\_lite\_wrapper.sv`
 - `xor\_permute\_top.sv`
 - `permute\_stage.sv`
3. Do not add the testbench for synthesis:
 - `tb\_xor\_permute.sv`
4. Open IP Integrator and create a new Block Design
5. Add `ZYNQ7 Processing System`
6. Run Block Automation for the Zynq PS
7. Add the custom RTL module `hdc\_axi\_lite\_wrapper`

8. Connect:

- ``s_axi_aclk`` to the PS AXI clock
- ``s_axi_aresetn`` to the AXI peripheral reset
- AXI slave interface to ``M_AXI_GP0`` from the Zynq PS

9. Run Connection Automation if Vivado offers it

10. Open Address Editor and note the assigned base address

11. Validate the block design

12. Create HDL wrapper

13. Run synthesis

14. Run implementation

15. Generate the bitstream

16. Export hardware to Vitis

Vitis Software Step

Use ``hdc_axi_example.c`` as the starting bare-metal application.

Before building, confirm the AXI base address in the source:

- Default example value: ``0x43C00000``
- Replace it with the actual Vivado-assigned base address if needed

Verification Status

The design was verified in ModelSim/Quarta with:

- Directed tests
- Randomized tests

- Backpressure checks
- Reset recovery checks
- Protocol assertions

Final verification result:

- `71 / 71` tests passed

Notes for Synthesis and Hardware Bring-Up

- `tb\_xor\_permute.sv` is simulation-only
- `.mpf`, `.cr.mti`, and `modelsim.ini` are not needed for synthesis
- The AXI wrapper is designed for simple PS-driven register access
- This is a good bring-up architecture before moving to AXI-Stream or DMA

Recommended Files to Copy to Linux

- `hdc\_axi\_lite\_wrapper.sv`
- `xor\_permute\_top.sv`
- `permute\_stage.sv`
- `hdc\_axi\_example.c`
- `HDC\_Project\_Guide.pdf`